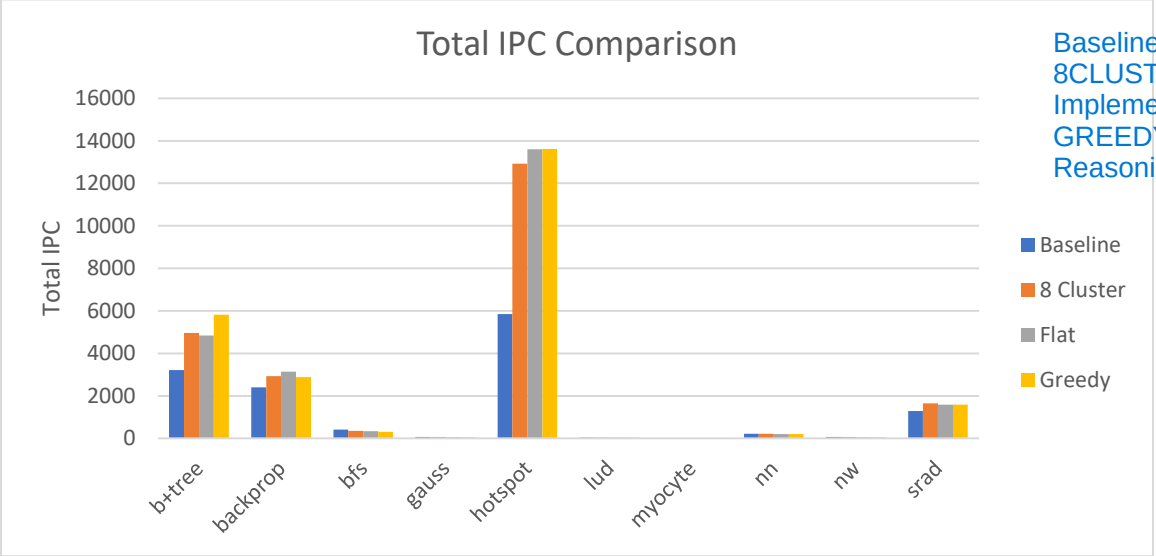
ECE60827 Hardware Simulation 2, 4/16/2023, Jerry Liu

Full Suite Statistics:

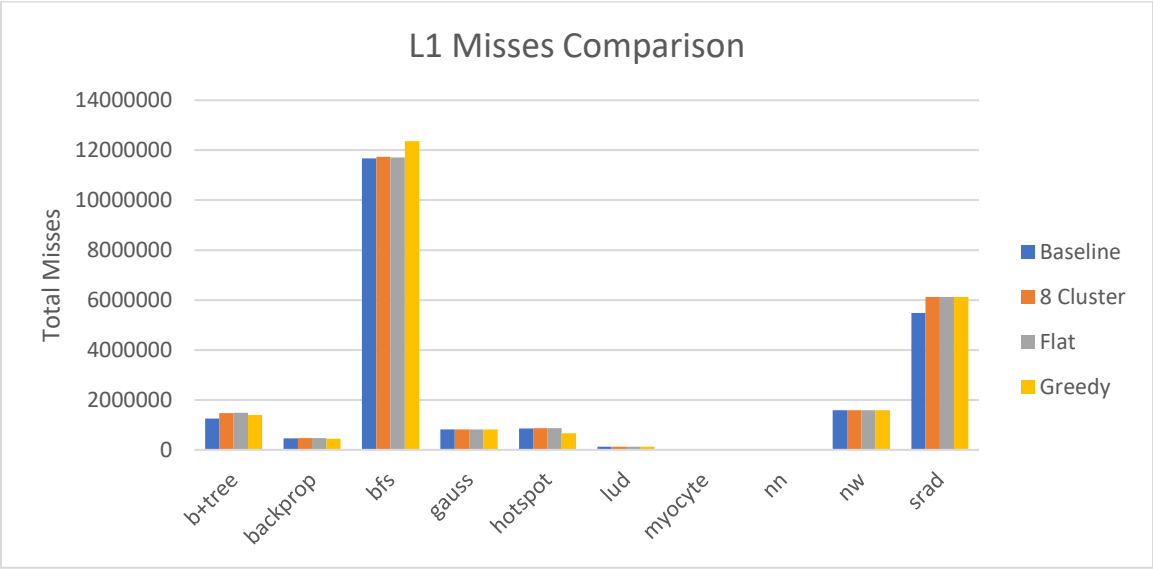
Total: 100/100

Baseline: 25/25
8CLUSTER: 25/25
Implementation of FLAT, GREEDY: 25/25
Reasoning of plots: 25/25



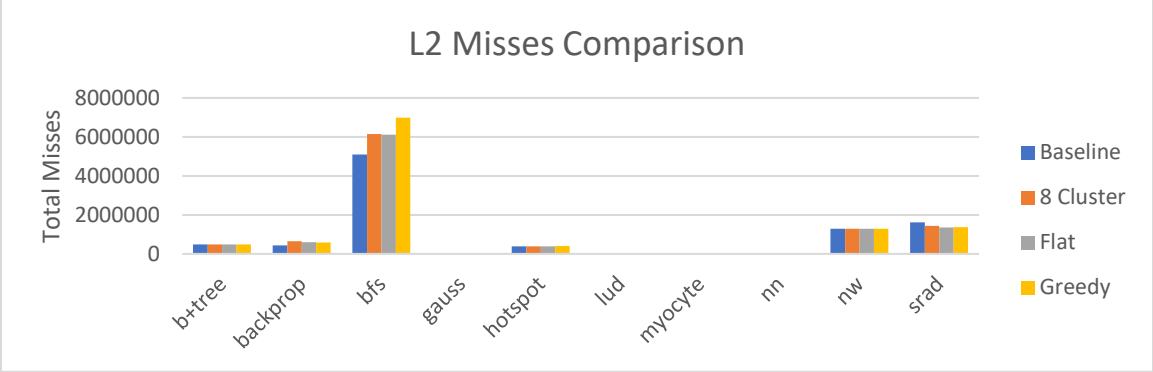
IPC Observations:

Based on the graph, it appears that all the (modified) simulations have a higher IPC than the baseline simulation. This is expected since all of them use the 8-cluster simulation as a base, and since that effectively increases the SM count by 8x, the IPC should increase as there are more cores available. As for the different schedulers, it seems that some benchmarks, like b+tree, are more favorable to the greedy scheduler, while others, like backprop, favor the flat scheduler. This is most likely caused by a difference in the behavior of the benchmarks. For example, the greedy scheduler could better exploit locality since it attempts to assign possibly relevant work to one SM instead of using a more round-robin approach.



L1(D) Misses Observations:

The different changes made to the simulator seem to have a relatively small effect on the total L1 misses for the test suite. Increasing the core concentration per cluster to 8 generally seems to increase misses for some benchmarks, which is probably caused by the additional thread strain on the caching system. In terms of the different CTA schedulers, the deviations noted could be explained once again by the difference in locality. For example, bfs could be less favorable to a greedy scheduler because schedules too large of a scope to one SM, which increases its miss rate. Conversely, this type of scheduling could be beneficial for hotspot because it allows the cache to better exploit the inter-block locality.



L2 Misses Observations:

The L2 miss statistics are similar to the L1 misses, but the differences are slightly more pronounced due to the filtering effect of the L1 cache. The most interesting statistic to note is that srad favors 8-cluster in terms of L1 misses, but favors baseline for L2 misses. This is likely explained by the additional strain of adding 8 cores to each cluster. While the L1 cache is private to each SM, the L2 is shared between the cores, so it makes sense that adding more cores increases the L2 miss rate.

Shader CTA Visualization Plots:

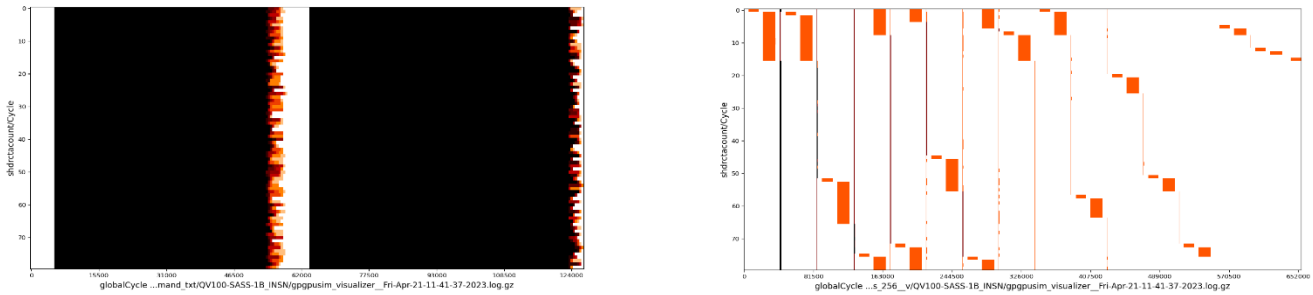


Figure 1, B+Tree (Left) and LUD (Right) Normal Baseline Plots

Baseline Observations:

From the baseline plots, it appears that b+tree has a two-phase computation, with each phase having enough work to be spread fully across all of the available cores. On the other hand, lud appears to be much more fragmented, with each step having less work available to be scheduled, which sometimes causes the core to not be fully utilized.

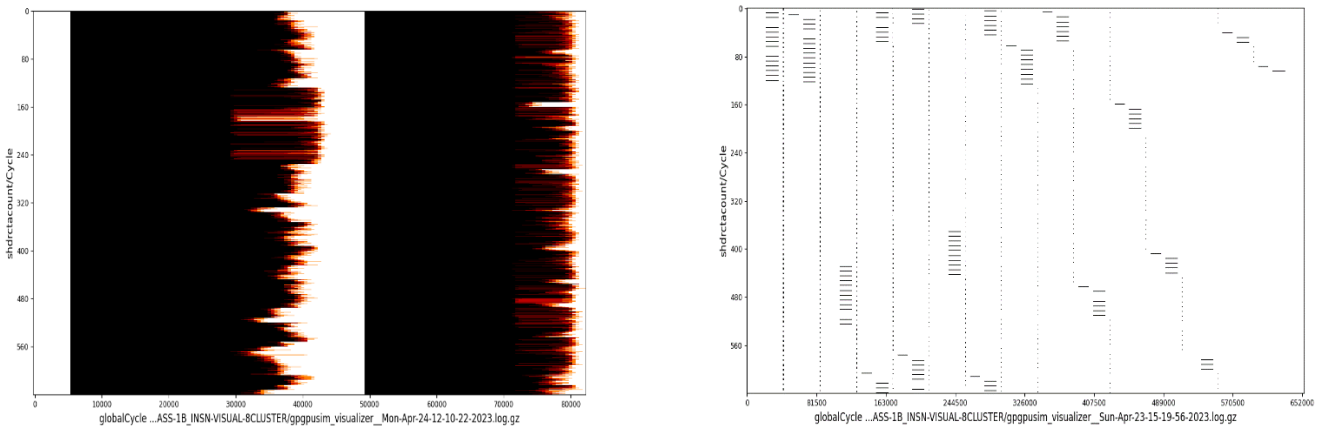


Figure 2, B+Tree (Left) and LUD (Right) 8 Cluster Plots

8-Cluster Observations:

Similar to the observations from the metrics section, the addition of 7 cores per cluster allows for much more work to be completed in parallel (if work is available). For b+tree, which had a lot of parallel computation, the plot remains relatively unchanged. However, lud did not have as much work to share across the cores, and as a result, the default round-robin scheduler is unable to fully utilize all the cores in the cluster, which causes gaps between each utilized SM.

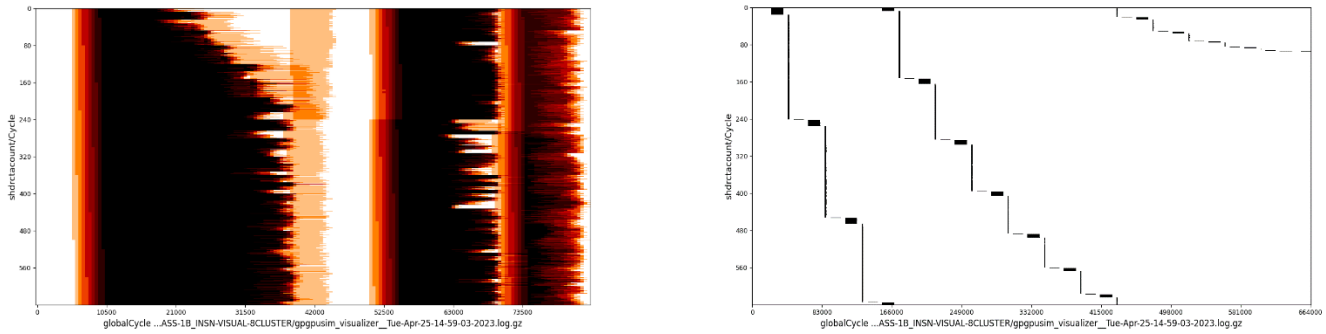


Figure 3, B+Tree (Left) and LUD (Right) Flat Plots

Flat Observations:

The flat scheduler essentially acts a round-robin across the SM's instead of the clusters. This is clearly visible in b+tree, where the workload starts low for each SM (scheduler assigns one CTA to each SM) before wrapping around and increasing. As for lud, the reduced workload is assigned in chunks, and no new work is assigned until each chunk is complete. Since the flat scheduler is implemented such that it remembers the last SM it assigned to, the lud plot shows a staircase pattern of SM utilization as it attempts to spread the relatively little work available across all the cores.

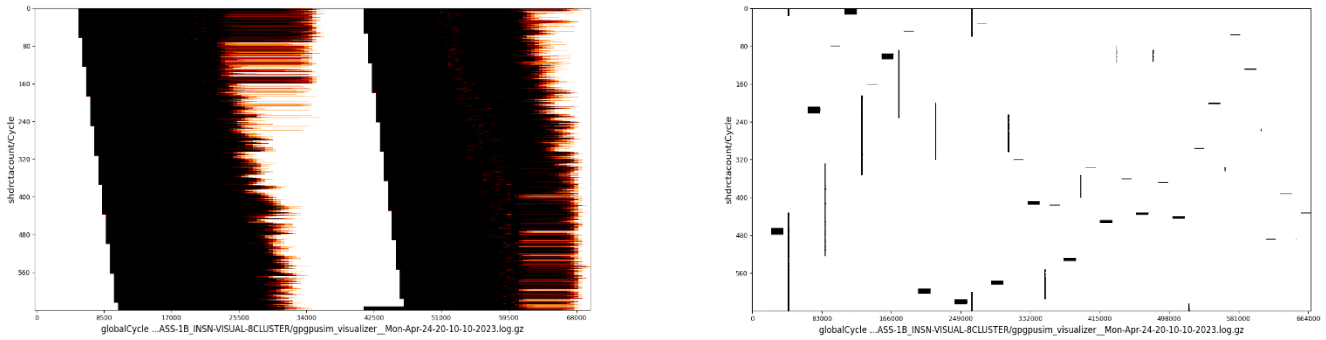


Figure 4, B+Tree (Left) and LUD (Right) Greedy Plots

Greedy Observations:

The greedy scheduler is implemented such that it attempts to pack as much work as possible onto a single SM before doing the same to other SM's in the same cluster. This behavior is clearly demonstrated in the b+tree plot. The first SM (and by extension cluster) is immediately assigned as much work as possible before subsequent SM's/clusters are assigned in later cycles. The scheduler then wraps around back to the first SM, and assigns the remaining work to the lower indexed SM's (unlike the flat scheduler which assigned the workload more evenly). On the other hand, while lud's plot is more chaotic and harder to read, it is easily explained by how the greedy scheduler is implemented. If the scheduler has no work to assign to a core/cluster (which is common for lud), or that cluster is full, it continuously cycles between the clusters looking for the next time it successfully assigns a new CTA. Once that occurs, the scheduler will "become" greedy and pack that core/cluster as much as possible. This means that the plot can (and does) look very fragmented between work cycles, with new cycles potentially starting from different cluster positions than the last. If the greedy scheduler did resume from the last cluster it assigned to, the plot would be identical to the flat scheduler because each CTA appears to fully occupy each SM. Aside from the fragmentation, we can observe that, when work is available, the scheduler packs the work into as few SM's as possible (as expected).